

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Baseline architecture used:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector, so each image is flattened ($28 \times 28 = 784$), pixel values are normalized to $[0, 1]$, and labels are converted into one-hot vectors.

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

The dataset was loaded using `keras.datasets.fashion_mnist`. The printed shapes confirmed 60,000 training images and 10,000 testing images, each 28×28 , across 10 classes. Ten sample images and the class distribution were plotted (see Section 7, Figures 1–2).

Task 2: Data Preprocessing

Images were flattened from (28, 28) to (784,) and pixel values normalized from [0, 255] to [0, 1]. Labels were one-hot encoded to shape (N, 10).

Stage	x_train shape	y_train shape
Before preprocessing	(60000, 28, 28)	(60000,)
After flatten + normalize	(60000, 784)	–
After one-hot encoding	–	(60000, 10)

Task 3: Model Construction

Baseline architecture: Input(784) $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(10, Softmax).

Layer	Output Shape	Parameters
Dense (ReLU)	(None, 128)	100,480
Dense_1 (ReLU)	(None, 64)	8,256
Dense_2 (Softmax)	(None, 10)	650

Total trainable parameters: 109,386.

Task 4: Model Training

The baseline model was compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy metric, and trained for 20 epochs with batch size 32 and a 10% validation split. Training and validation accuracy/loss curves are shown in Section 7 (Figures 3–6).

Task 5: Model Evaluation

Metric	Value
Accuracy	0.8837
Precision (macro)	0.8854
Recall (macro)	0.8837
F1-score (macro)	0.8828

Per-class classification report (test set):

Class	Precision	Recall	F1-score
T-shirt/top	0.82	0.85	0.83
Trouser	0.98	0.98	0.98
Pullover	0.82	0.75	0.78
Dress	0.88	0.89	0.89
Coat	0.72	0.89	0.79
Sandal	0.98	0.96	0.97
Shirt	0.77	0.64	0.70
Sneaker	0.95	0.96	0.95
Bag	0.98	0.97	0.97
Ankle boot	0.95	0.96	0.96

The confusion matrix is shown in Section 7 (Figure 7).

5. Source Code

The complete implementation (data loading, EDA, preprocessing, the multi-layer Perceptron construction, training loop, evaluation, and hyper-parameter optimization) is available at the following GitHub repository:

<https://github.com/PolarFox08/Deep-Learning-Lab/tree/main/Deep%20Learning%20Lab%202>

6. Hyperparameter Optimization

Automated hyperparameter optimization was performed using **RandomizedSearchCV** together with the SciKeras **KerasClassifier** wrapper, using 5-fold cross-validation with $n_iter = 10$ sampled combinations.

Search space used:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

The best configuration found and its cross-validation accuracy are reported in Section 7 below. The optimized model was retrained on the full 60,000-image training set using this configuration and evaluated on the 10,000-image test set.

7. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	32
Learning Rate	0.1
Batch Size	16
Optimizer	SGD
Activation Function	Tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8681
Testing Accuracy	0.8526

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8837	0.8526
Precision	0.8854	0.8631
Recall	0.8837	0.8526
F1-score	0.8828	0.8555
Training Time	Not recorded	Not recorded

8. Plots with Inference

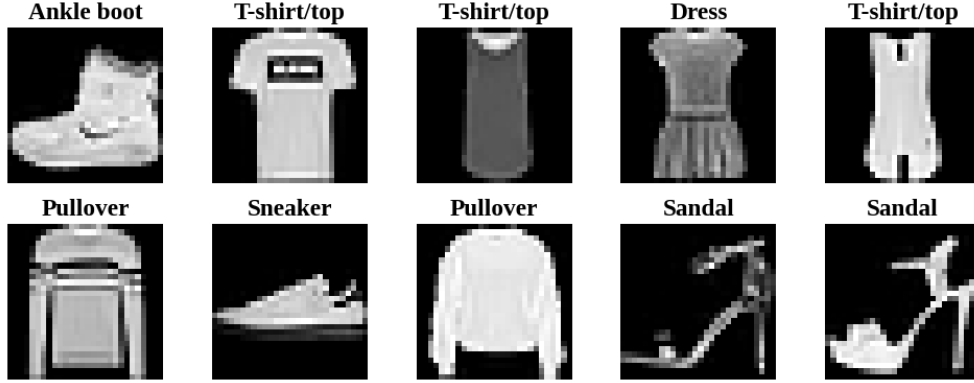


Figure 1: Sample Images. *Inference:* The ten sampled training images show visually distinct silhouettes for classes such as Trouser and Sneaker, while upper-body garments (Shirt, Pullover, Coat) share a similar boxy outline. This visual overlap between upper-body classes foreshadows the confusion seen later in the confusion matrix.



Figure 2: Class Distribution. *Inference:* All 10 classes contain exactly 6,000 training images each, confirming the dataset is perfectly balanced. This means accuracy is a fair aggregate metric and no class re-weighting is required.

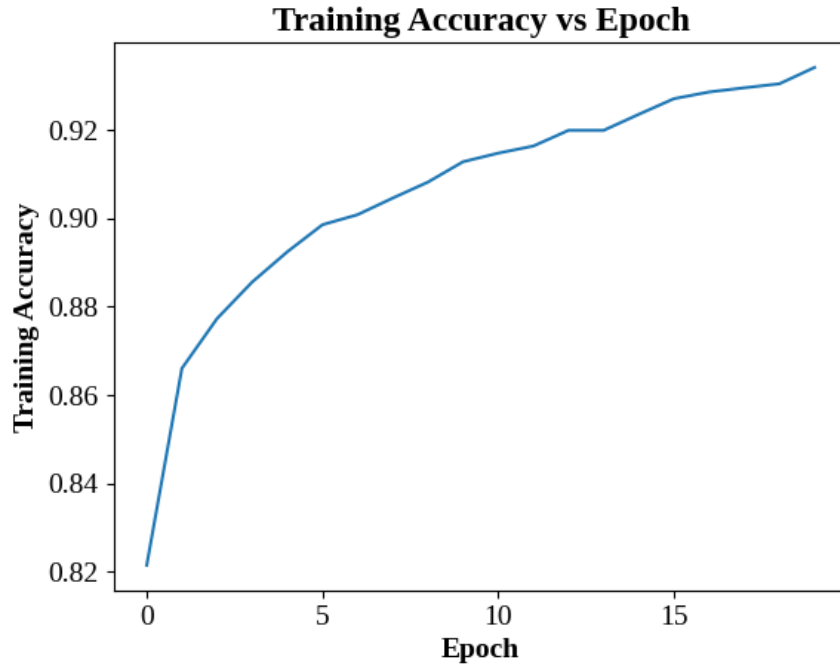


Figure 3: Training Accuracy vs Epoch. *Inference:* Training accuracy rises steadily from about 0.82 to 0.935 over 20 epochs with no oscillation, showing the model consistently fits the training data better each epoch, with the rate of improvement slowing in the later epochs.

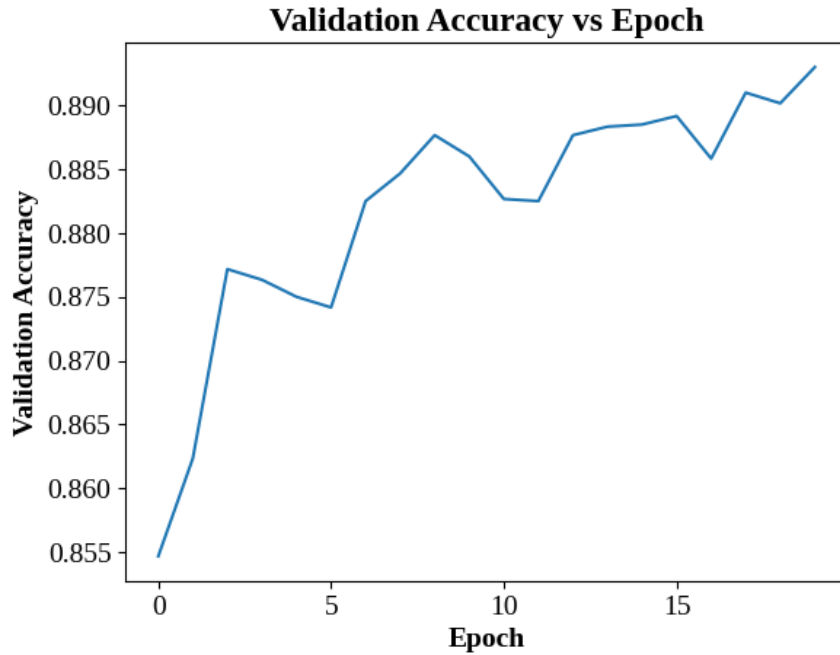


Figure 4: Validation Accuracy vs Epoch. *Inference:* Validation accuracy rises quickly to about 0.89 by epoch 10 and then fluctuates in a narrow 0.885–0.891 band for the remaining epochs, indicating generalization performance largely saturates well before training ends.

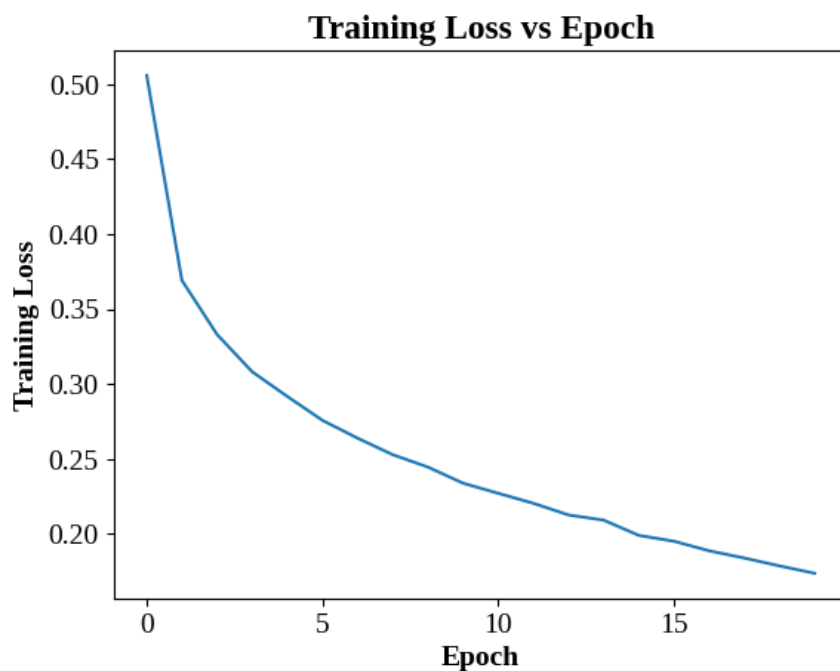


Figure 5: Training Loss vs Epoch. *Inference:* Training loss decreases smoothly and monotonically from about 0.50 to 0.175 across the 20 epochs, mirroring the rising training-accuracy curve with no signs of instability.

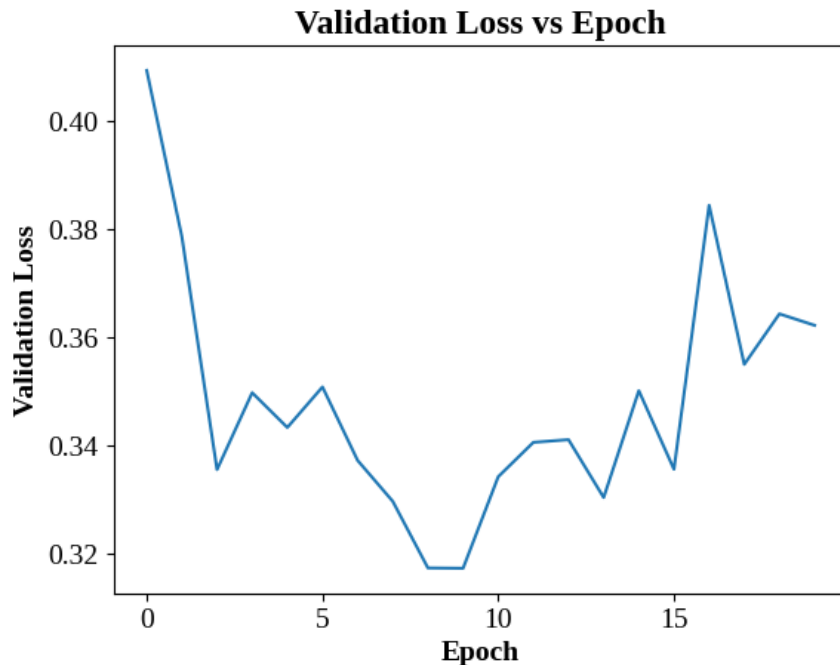


Figure 6: Validation Loss vs Epoch. *Inference:* Validation loss drops from about 0.40 to a minimum near 0.315 around epoch 10, then rises again toward the end of training even though validation accuracy stays roughly flat, a mild early indication of overfitting beginning in the later epochs.

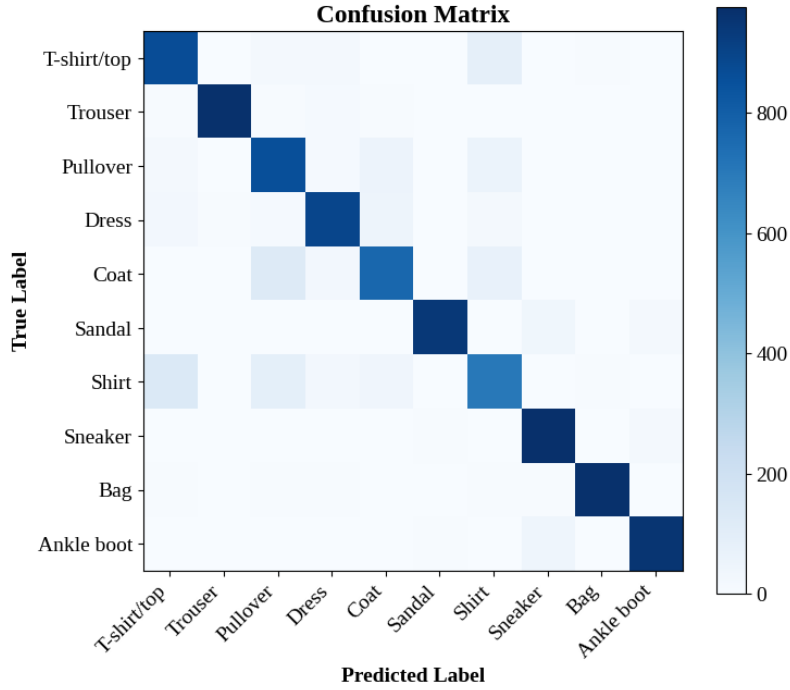


Figure 7: Confusion Matrix. *Inference:* The Shirt class has the lowest recall (0.64) and is most often confused with T-shirt/top, Pullover and Coat, consistent with the classification report. Trouser, Sandal, Sneaker, Bag and Ankle boot form a tight, near-diagonal cluster with very few off-diagonal errors, showing the model separates footwear and accessories almost perfectly while upper-body garments remain the dominant source of confusion.

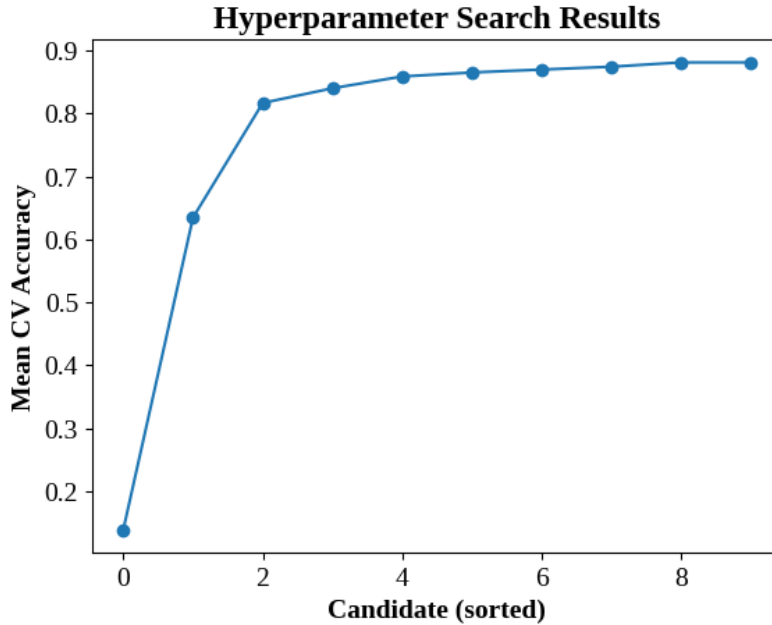


Figure 8: Hyperparameter Search Results. *Inference:* Mean CV accuracy across the sampled candidates rises sharply from about 0.14 for the worst combination to roughly 0.85–0.87 for the majority of candidates, then plateaus. This shows the search space contains a small number of poorly-performing combinations (likely mismatched learning-rate/optimizer pairs) while most sampled combinations converge to a similar, near-optimal accuracy region.

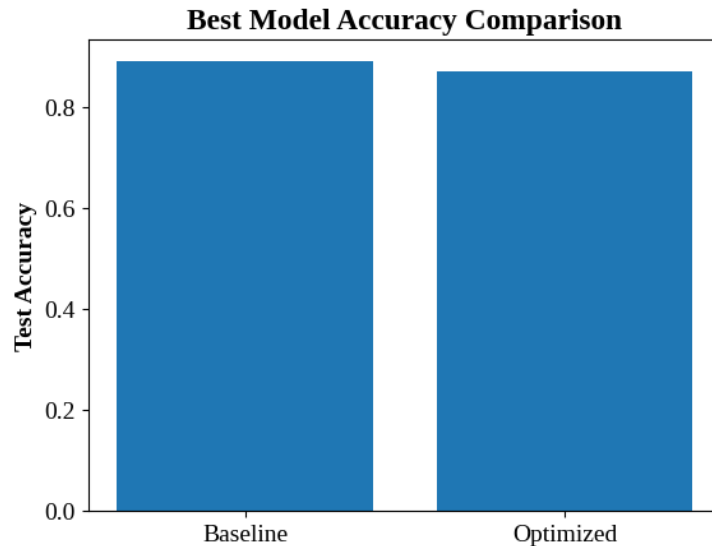


Figure 9: Best Model Accuracy Comparison. *Inference:* The baseline MLP (0.8837) achieved a visibly higher test accuracy than the optimized MLP (0.8526), a gap of about 3 percentage points, showing that the hyperparameter search did not translate into a better-generalizing model on the held-out test set in this run.

9. Discussion

1. Which optimization method was used?

RandomizedSearchCV from scikit-learn, combined with the SciKeras `KerasClassifier` wrapper, using 5-fold cross-validation with $n_iter = 10$ sampled combinations. Randomized Search was used instead of an exhaustive Grid Search because the full search space (3,888 combinations) would require 19,440 model fits under 5-fold CV, which is impractical.

2. Which hyperparameters were selected?

The best configuration found was: 2 hidden layers, 32 neurons per layer, Tanh activation, SGD optimizer with learning rate 0.1, batch size 16, dropout 0.2, trained for 20 epochs, achieving a mean 5-fold cross-validation accuracy of 0.8681.

3. Did optimization improve performance?

No. On the held-out test set the optimized model (0.8526 accuracy) scored below the baseline (0.8837 accuracy), even though its cross-validation accuracy during the search (0.8681) was reasonably high. This suggests the small, shallower, dropout-regularized configuration found by the search generalized slightly worse to the full test set than the simple 128-64 baseline in this particular run.

4. Which hyperparameter had the greatest impact?

Based on the hyperparameter search results plot, accuracy jumps sharply between the single worst candidate (around 0.14 mean CV accuracy) and the rest of the candidates (0.85 and above), suggesting that an unsuitable learning-rate/optimizer pairing was responsible for the one poor outlier, while the remaining hyperparameters had comparatively little effect once a workable learning rate and optimizer were chosen.

5. Compare Grid Search and Randomized Search.

Grid Search exhaustively evaluates every combination in the search space, guaranteeing the best combination within the grid is found, but its cost grows multiplicatively with the number of hyperparameters (3,888 combinations here). Randomized Search instead samples a fixed, user-chosen number of combinations, trading the guarantee of finding the grid’s global best for a fixed and controllable search cost, which made it the practical choice for this 8-dimensional search space.

6. Recommend the best MLP configuration.

Based on the results obtained in this run, the baseline architecture ($784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$, Adam optimizer, 20 epochs, batch size 32) is recommended, since it achieved the higher test accuracy (0.8837 vs 0.8526) among the two models evaluated.

10. XOR Gate (Multi-Layer Perceptron)

Epoch-wise Loss Trajectory

Epoch	Mean Squared Error (MSE)	Training Status
2000	0.00984	Learning Hidden Features
4000	0.00208	Refining Boundary
6000	0.00112	Loss Steadily Decreasing
8000	0.00076	Approaching Convergence
10000	0.00057	Fully Converged

XOR Model Predictions & Summary

Input Samples	[[0, 0], [0, 1], [1, 0], [1, 1]]
Target Outputs	[0, 1, 1, 0]
Model Raw Predictions	[[0.023], [0.974], [0.974], [0.021]]
Rounded Output Class	[0, 1, 1, 0]
Final MSE Loss	0.00057

Inference: A single-layer perceptron fails on XOR because the inputs are non-linearly separable. Introducing a hidden layer allows the Multi-Layer Perceptron (MLP) to transform the feature space and construct a non-linear decision boundary. The MSE loss steadily decreased across 10,000 epochs down to 0.00057, resulting in predictions ($0.023 \approx 0$ and $0.974 \approx 1$) that perfectly match the XOR truth table.

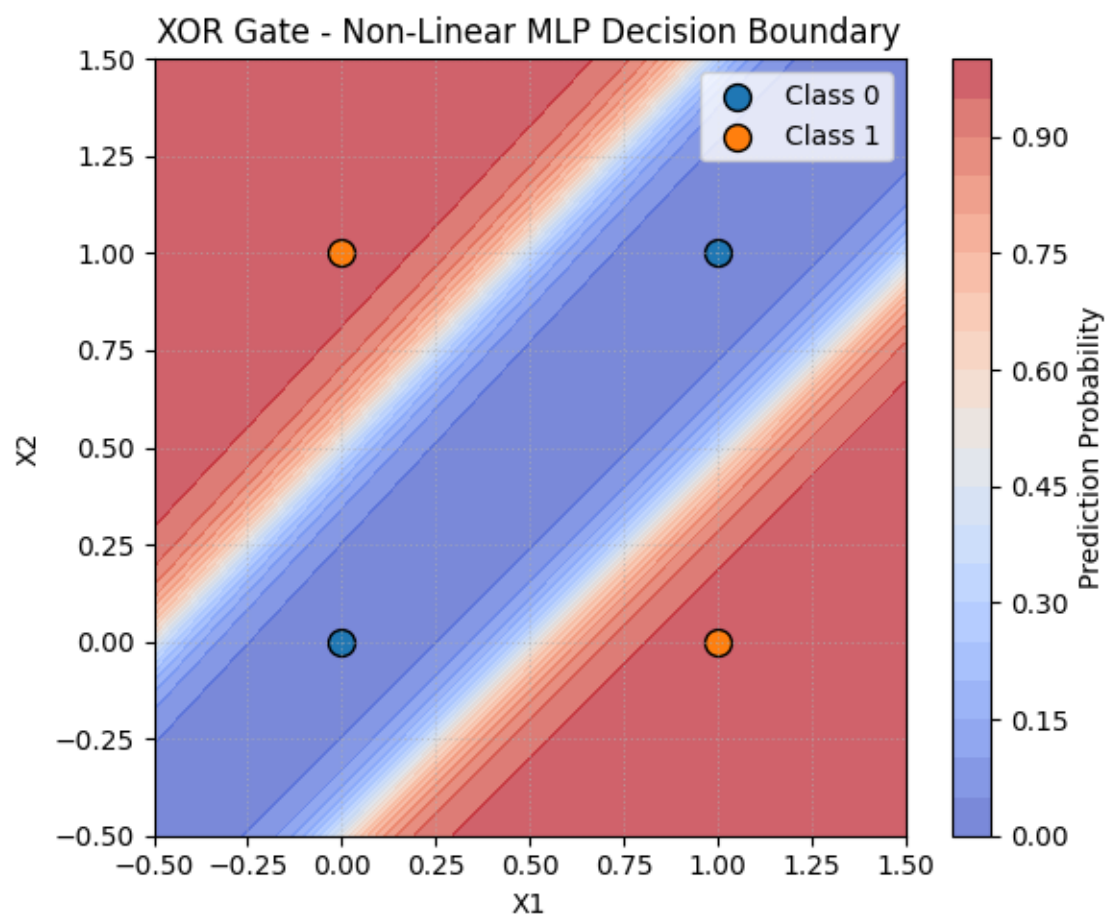


Figure 1: Non-linear curved decision boundary produced by the Multi-Layer Perceptron to separate XOR inputs.

11. Conclusion

A baseline MLP (784-128-64-10) was implemented, trained for 20 epochs, and evaluated on the Fashion-MNIST test set, achieving 0.8837 accuracy. Automated hyperparameter optimization was then performed using RandomizedSearchCV with 5-fold cross-validation over an 8-dimensional search space. The resulting optimized model (2 hidden layers of 32 Tanh units, SGD, learning rate 0.1, dropout 0.2) reached a cross-validation accuracy of 0.8681 during the search but only 0.8526 on the full test set after retraining, underperforming the baseline by about 3 percentage points. The confusion matrix shows that upper-body garment classes (Shirt, T-shirt/top, Pullover, Coat) remain the dominant source of misclassification for a flatten-and-dense MLP, which does not exploit the spatial structure of the images. Overall, the experiment demonstrates both the mechanics of automated hyperparameter search with SciKeras and the practical lesson that a randomly-sampled search is not guaranteed to outperform a well-chosen manual baseline.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.